P1453R0
Modularizing the Standard Library is a Reorganization Opportunity

Published Proposal, 2019-01-21

Author:
Bryce Adelstein Lelbach (NVIDIA)

Source:
GitHub

Issue Tracking:
GitHub

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:
LEWG

Table of Contents

References

Informative References

§ 1. Introduction

Modules are coming to C++. After the San Diego 2018 C++ committee meeting, it seems quite likely that we will have modules in C++20. This feature will have a transformative impact on almost all C++ code. Consider the typical "hello world" program in C++:

```
#include <iostream>
int main() { std::cout << "hello world\n"; }
```

"hello world" has remained constant from C++98 through C++17. With C++20, however, it may finally change:

```
import /* ??? */;
int main() { std::cout << "hello world\n"; }
```

But, what will `/* ??? */` be?

While we appear to have achieved consensus on a design for the modules language feature, our plan for how and when the C++ standard library will be modularized are not as mature. Some proposals have been made ([P0581R1] and [P1212R0]) and preliminary discussions have taken place ([2018-Jacksonville-LEWG-P0581R0-Minutes] and [2018-San-Diego-EWG-P1212R0-Minutes]), but we haven't committed to a path yet. Given that the C++20 cycle is nearly over, it's time for us to make a decision on our strategy for standard library modules in C++20.

§ 2. The Need for Reorganization

Modules are a sorely needed opportunity to reorganize the standard library. Today's header-based partitioning of the standard library has not aged well over the past two decades. Some of the pain points include:

— **Transitive Includes:** Standard library headers are allowed to include other standard library headers. There is often implementation divergence in the use of transitive includes. When users rely on transitive includes, they can end up with non portable code.

— **Overly Coarse Includes:** Some standard library headers have evolved into repositories of facilities that do not necessarily belong together. For example, in C++98, `<functional>` was a simple header with a simple mission: provide unary and binary function objects for operators.

Including `<functional>` with GCC 4.1 compiling in C++03 mode added 600 lines to your code. Today, `<functional>` contains many things, including `std::function` (which needs type traits and memory management facilities) and `std::*_searchers` (which need string processing facilities). Including `<functional>` with GCC 8 compiling in C++11 mode adds 44,691 lines to your code. That's a lot if all you needed was `std::plus`.

— **Lack of Logical Partitioning:** It is common for C++ programmers to think of each standard library header as encapsulating a component of the standard library. For example, the regex library lives in `<regex>`, the filesystem library lives in `<filesystem>`, and `std::tuple` lives in `<tuple>`. However, many components of the standard library are spread across multiple headers, such as the sequence algorithms. Most sequence algorithms are found in `<algorithm>`, but some are found in `<numeric>` and `<memory>`. This has even been confusing to the committee, which has often forgot to apply changes applicable to all algorithms to the ones in `<numeric>` and `<memory>`. We even moved the section in the standard for the `<numeric>` header into the `[algorithms]` clause recently so that it will no longer be overlooked. Additionally, some standard library facilities that live in omnibus headers and can be difficult to locate. Where is `std::pair`? In `<utility>`, not `<pair>`. Where is `std::unique_ptr`? In `<memory>`, not `<unique_ptr>`.

— **Freestanding:** There is general agreement on the committee that the current definition of the freestanding standard library subset is not particularly useful. At the same time, there are a number of constituencies (embedded, kernel/OS development, parallel programming) that desire a useful freestanding standard library and would like to see freestanding overhauled in a future standard ([P0829R3], [P1212R0], [P1376R0]).

All of these issues could be solved by a careful reorganization of the standard library. The only reason we have not undertaken such a reorganization has been a lack of opportunity before modules.

§ 3. A Rare Opportunity

This opportunity to reorganize the standard library is a rare one. Standard library implementations make strong backwards compatibility guarantees, so we cannot just decide to move entities from one header to another.

Modules, however, move us away from a header-based model entirely. When we modularize the standard library, we are not bound to the existing structure of our current header-based model. Thee moduarlized standard library can be repartition into more sensible subsets.

However, we only have one chance to do this. Once we ship a modularized standard library, we will find ourselves in the same situation that we have today. We will be unable to reorganize things without

breaking backwards compatibility.

This may be our only chance in the lifetime of C++ to reorganize the standard library. Thus, it is imperative that we get it right.

§ 3.1. Time is Limited

We are nearing the end of the C++20 development cycle. We plan to ship a committee draft of the C++20 standard six months from the authoring of this paper.

Integrating new language features into the standard library is one of the harder parts of our standardization work. Recently, we have wisely decided to exercise caution when rolling out new language features in the standard library.

Take concepts as an example. In C++20, we have a core concepts library, and a major new piece of the standard library, ranges, uses concepts. However, we have not attempted to go through the entire standard library and deploy concepts everywhere we can for C++20. Additionally, we have been conservative in the use of concepts for new library features for C++20. As far as this author is aware, only the new ranges library will use concepts. It seems likely that in future standards, in places where proper constrained template parameters are desirable, we will either update existing facilities to use concepts or introduce new improved versions of those facilities.

While it may seem counter-intuitive for the standard library to not aggressively embrace new language features, it is in fact the right thing to do in many cases. While the impact of new language feature on the library should be considered and evaluated, we should neither make new language features wait for deployment within the standard library nor rush that deployment to not delay the feature. When a language feature is ready, so long as we are confident that we will be able to complete deployment of the feature in the standard library in the future, we should be ready to ship it. We should not let the perfect be the enemy of the good.

§ 4. Options for a C++20 Modular Standard Library

§ 4.1. Don’t Modularize the Standard Library in C++20

The first and most conservative option would be to not modularize the standard library in C++20. Users would access the standard library via legacy imports or coontinue `#includes`.

Pros: This approach preserves the most freedom for future modularization and presents no risk of shipping something we are unable to change later.

Cons: The lack of a modularized standard library may inhibit adoption of modules or lead to implementation-specific standard library modules.

§ 4.2. Modularize the Existing Structure in C++20 and Reorganize Later

In this approach, we would introduce standard library modules in C++20 that map to the existing standard library headers. For each standard library header `<foo>`, we would introduce a `std.header.foo` module.

The use of the `std.header` prefix is intended to reserve syntactic space for the future. For example, we might want to have a `std.algorithm` module in the future which contains all the sequence algorithms, while the `std.header.algorithm` module would contain just the sequence algorithms in `<algorithm>`. This prefix could be spelled differently, possibly `std.legacy`, `std.old`, or `std.v1`.

Pros: This approach provides standard library modules in C++20, preserves a great deal of freedom for future modularization, and presents an acceptably small amount of risk of shipping something we are unable to change later.

Cons: The `std.header` modules we'd be shipping would be difficult to get rid of. They'd co-exist with whatever new standard library modules we introduce in the future and potentially preserve some of the problems with the existing standard library structure identified in §2 The Need for Reorganization.

There is one major question with this approach. Will we be able to move the definition of an entity between modules without breaking ABI compatibility? E.g. If entity `foo` is currently defined in `std.header.bar`, in the future can we introduce a new module `std.foo` which defines `foo`, and modify `std.header.bar` to simply re-export `foo`?

§ 4.3. One Big Module for C++20

One option would be to introduce a `std` module that includes everything in the standard library. For finer grained usage, users could either use the modules from §4.2 Modularize the Existing Structure in C++20 and Reorganize Later, use legacy imports, or continue using `#includes`.

Pros: This would be easy to use and would still allow us to reorganize the standard library into finer grained modules in the future.

Cons: The standard library has a number of global dynamic constructors (such as the ones in iostreams) and auxiliary dependencies (filesystem libraries, regex libraries, etc). It would be undesirable to encourage people to pay for these as they may not be using them.

As with §4.2 Modularize the Existing Structure in C++20 and Reorganize Later, this approach is only sensible if we are able to move the definition of a standard library entity from one module to another without breaking ABI compatibility.

§ 4.4. Attempt to Reorganize the Entire Standard Library in C++20

In this approach, we would use the remaining time in the C++20 cycle to reorganize the structure of the standard library to address the problems identified in §2 The Need for Reorganization. This effort would likely need to include a redefinition of the freestanding standard library subset, as this would likely be our only chance to do so. So, this would probably be a combination of [P0581R1], [P0829R3], and [P1376R0].

Pros: We would have a complete solution in C++20 instead of a middleground approach with limitations.

Cons: This option has significant schedule risk, limited time for us to obtain field experience, closes the door for any post C++20 reorganization, and presents the highest risk of shipping something that we are unable to change later.

§ References

§ Informative References

[2018-Jacksonville-LEWG-P0581R0-Minutes]
2018 Jacksonville C++ Committee Meeting - LEWG P0581R0 Minutes. URL: http://wiki.edg.com/bin/view/Wg21jacksonville2018/P0581

[2018-San-Diego-EWG-P1212R0-Minutes]
2018 San Diego C++ Committee Meeting - EWG P1212R0 Minutes. URL: http://wiki.edg.com/bin/view/Wg21sandiego2018/P1212R0-San18

[P0581R1]
Marshall Clow, Beman Dawes, Gabriel Dos Reis, Stephan T. Lavavej, Billy O'Neal, Bjarne Stroustrup, Jonathan Wakely. Standard Library Modules. 11 February 2018. URL: https://wg21.link/p0581r1

[P0829R3]

Ben Craig. Freestanding Proposal. 6 October 2018. URL: https://wg21.link/p0829r3

[P1212R0]

Ben Craig. Modules and Freestanding. 6 October 2018. URL: https://wg21.link/p1212r0

[P1376R0]

Ben Craig. Summary of freestanding evening session discussions. 24 November 2018. URL: https://wg21.link/p1376r0